

1. Hybrid Inheritance

Hybrid Inheritance is a combination of two or more types of inheritance, such as single, multilevel, and hierarchical inheritance. Java does not support hybrid inheritance through classes because it can create ambiguity (Diamond Problem). However, it can be achieved using interfaces.

Example

```
interface A {  
    void show();  
}  
  
interface B {  
    void display();  
}  
  
class C implements A, B {  
  
    public void show() {  
        System.out.println("Method from Interface A");  
    }  
  
    public void display() {  
        System.out.println("Method from Interface B");  
    }  
}  
  
public class HybridInheritance {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.show();  
        obj.display();  
    }  
}
```

Output

```
Method from Interface A  
Method from Interface B
```

2. Interface

An Interface in Java is a blueprint of a class that contains abstract methods and constants. It is used to achieve 100% abstraction and multiple inheritance.

Key Points

- Declared using interface keyword.

- Methods are public and abstract by default.
- Variables are public, static, and final by default.
- A class uses implements keyword to inherit an interface.

Example

```
interface Animal {
    void sound();
}

class Dog implements Animal {

    public void sound() {
        System.out.println("Dog barks");
    }
}

public class InterfaceExample {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
```

Output

Dog barks

3. Super Keyword

The super keyword refers to the immediate parent class object. It is used to:

1. Access parent class variables.
2. Call parent class methods.
3. Call parent class constructor.

Example

```
class Parent {
    String name = "Parent Class";

    Parent() {
        System.out.println("Parent Constructor");
    }

    void display() {
        System.out.println("This is Parent Method");
    }
}
```

```

class Child extends Parent {
    String name = "Child Class";

    Child() {
        super(); // Calls Parent Constructor
        System.out.println("Child Constructor");
    }

    void show() {
        System.out.println(super.name); // Parent variable
        super.display(); // Parent method
    }
}

public class SuperKeyword {
    public static void main(String[] args) {
        Child obj = new Child();
        obj.show();
    }
}

```

Output

```

Parent Constructor
Child Constructor
Parent Class
This is Parent Method

```